

LDAP: Understanding the Active Directory Database From Kali

Delayed Recall Verification

What does TCP/445 tell an assessor?

Port 445 tells an assessor that the host is offering SMB services.

What's the difference between authentication, share visibility, and authorization?

Proving identity/who you are vs. what shares can you see exist vs. what permissions are you granted once authenticated

Why could Alice see finance in the share listing but still receive ACCESS_DENIED when trying to use it?

Because, share visibility vs authorization

What does @192.168.1.236 mean in a dig query?

Send query to dns server at

A record vs SRV record?

A record is the dns mapping (ip address to [google.com](https://www.google.com)) and SRV record is asking which server provides this service and on what port?

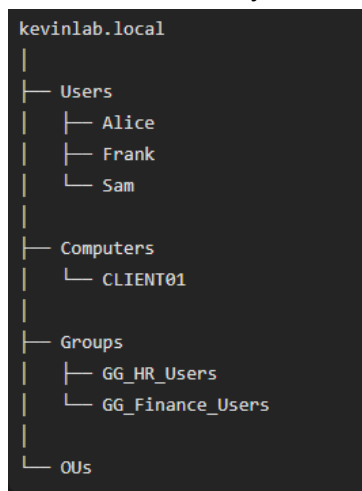
AXFR:

DNS zone transfer, "give me a copy of all dns records in this zone"

LDAP Theory

LDAP = Lightweight Directory Access Protocol

If we think about my AD build:



That information has to be stored and accessed somehow. AD DS maintains the directory.

LDAP is one of the protocols used to query and interact with that directory.

Active Directory = directory / database of enterprise identity information

LDAP = protocol used to ask that directory questions

Examples:

- Give me information about this user.
- What groups exist?
- Which users are members of this group?
- What computers exist?
- Which OU contains this object?

Distinguished Names

LDAP needs a way to identify objects within the directory hierarchy. Something may resemble:
CN=Alice, HR, OU=HR, OU=KEVINLAB-Users, DC=kevinlab, DC=local

Read it from the object outward:

1. CN=Alice HR
2. OU=HR
3. OU=KEVINLAB-Users
4. DC=kevinlab.local

CN = Common Name, OU we know, DC = Domain Component

Kevinlab.local becomes: DC=kevinlab, DC=local

Base DN

When querying LDAP, you need to establish: Where in the directory should my search begin?

My domain's base is: DC=kevinlab, DC=local

Conceptually: Start here -> kevinlab.local directory -> Search beneath it

That is a Base DN.

LDAP Attributes

Objects contain attributes. A user could have information such as: sAMAccountName, distinguishedName, memberOf, objectClass

SO LDAP isn't simply: "Give me Alice." It can return structured properties describing Alice.

Authentication / Bind

Before LDAP lets us retrieve particular information, identity matters. LDAP uses the concept of a bind to establish an authentication context.

Kali -> LDAP connection -> Who are you? -> Bind/authentication -> Directory access according to permissions

Why an assessor cares

Suppose an ordinary domain account can legitimately query directory information

An assessor may learn: Domain Structure, Users, Groups, Computers, Group Memberships, Service Accounts, Naming Conventions

Guided Practical: Query Directory

Authorized target: SRV-DC01, kevinlab.local

Verify current IP address. (192.168.1.236 for SRV-DC01)

Confirm LDAP

Just need to ask nmap about port 389 (specific to LDAP)...is it reachable?

```
(nigist@kali)-[~]
$ nmap -Pn -p 389 192.168.1.236
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-27 23:07 EDT
Nmap scan report for 192.168.1.236
Host is up (0.0031s latency).

PORT      STATE SERVICE
389/tcp    open  ldap

Nmap done: 1 IP address (1 host up) scanned in 0.06 seconds
```

Meet ldapsearch

On Kali: ldapsearch is a common command-line LDAP querying utility

General structure: ldapsearch [connection/authentication options] [search options]

Ask for RootDSE information

There's a special LDAP entry called RootDSE. Think of it as: "Tell me basic information about this directory server itself."

Try: **ldapsearch -x \ -H ldap://192.168.1.236 \ -s base \ -b "" \ defaultNamingContext namingContexts dnsHostName**

-x: use simple authentication behavior rather than SASL for this query

-H: URI of the LDAP service

-s base: search only the base object itself

-b "": an empty base DN here is used to query RootDSE

Requested attributes: defaultNamingContext, namingContexts, dnsHostName

```

└─$ ldapsearch -x -H ldap://192.168.1.236 -s base -b "" defaultNamingContext
namingContexts dnsHostName
# extended LDIF
#
# LDAPv3
# base <> with scope baseObject
# filter: (objectclass=*)
# requesting: defaultNamingContext namingContexts dnsHostName
#
#
dn:
dnsHostName: SRV-DC01.kevinlab.local
namingContexts: DC=kevinlab,DC=local
namingContexts: CN=Configuration,DC=kevinlab,DC=local
namingContexts: CN=Schema,CN=Configuration,DC=kevinlab,DC=local
namingContexts: DC=DomainDnsZones,DC=kevinlab,DC=local
namingContexts: DC=ForestDnsZones,DC=kevinlab,DC=local
defaultNamingContext: DC=kevinlab,DC=local

# search result
search: 2
result: 0 Success

# numResponses: 2
# numEntries: 1

```

defaultNamingContext: DC=kevinlab,DC=local....the server has told us: this is the directory naming context for your AD domain, that gives us the LDAP search base...DNS enumeration gave us the LDAP server, LDAP RootDSE gives us the directory structure

Can anonymous access search the domain?

Now trying to use: DC=kevinlab,DC=local as the search base without providing credentials...what will the server disclose without authentication?

```

(nigist@kali)-[~]
└─$ ldapsearch -x -H ldap://192.168.1.236 -b "DC=kevinlab,DC=local" -s base "
(objectClass=*)"
# extended LDIF
#
# LDAPv3
# base <DC=kevinlab,DC=local> with scope baseObject
# filter: (objectClass=*)
# requesting: ALL
#
# search result
search: 2
result: 1 Operations error
text: 000004DC: LdapErr: DSID-0C090A58, comment: In order to perform this ope
ra
tion a successful bind must be completed on the connection., data 0, v4f7c
# numResponses: 1

```

Anonymous access does not work, need the binding which is the authentication process.

Authenticate as a normal user

Going to try using Alice now. What can a normal authenticated domain identity discover?

Command: **ldapsearch -x -H ldap://192.168.1.236 -D 'ahr@kevinlab.local' -W -b 'DC=kevinlab,DC=local' '(objectClass=user)' sAMAccountName distinguishedName**

-D: identity used to bind

-W: prompt for the password

(objectClass=user): ask for user objects

Attributes: sAMAccountName, distinguishedName

REMEMBER: I gave ALICE a new password!

```
└─$ ldapsearch -x \
-H ldap://192.168.1.236 \
-D 'ahr@kevinlab.local' \
-W \
-b 'DC=kevinlab,DC=local' \
'(objectClass=user)' \
sAMAccountName distinguishedName
Enter LDAP Password:
# extended LDIF
#
# LDAPv3
# base <DC=kevinlab,DC=local> with scope subtree
# filter: (objectClass=user)
# requesting: sAMAccountName distinguishedName
#
# Administrator, Users, kevinlab.local
dn: CN=Administrator,CN=Users,DC=kevinlab,DC=local
distinguishedName: CN=Administrator,CN=Users,DC=kevinlab,DC=local
sAMAccountName: Administrator
# Guest, Users, kevinlab.local
dn: CN=Guest,CN=Users,DC=kevinlab,DC=local
distinguishedName: CN=Guest,CN=Users,DC=kevinlab,DC=local
sAMAccountName: Guest
# SRV-D01, Domain Controllers, kevinlab.local
dn: CN=SRV-D01,OU=Domain Controllers,DC=kevinlab,DC=local
distinguishedName: CN=SRV-D01,OU=Domain Controllers,DC=kevinlab,DC=local
sAMAccountName: SRV-D01$
# krbtgt, Users, kevinlab.local
dn: CN=krbtgt,CN=Users,DC=kevinlab,DC=local
distinguishedName: CN=krbtgt,CN=Users,DC=kevinlab,DC=local
```

```
# Kevin KA. Admin, IT, KEVINLAB-Users, kevinlab.local
dn: CN=Kevin KA. Admin,OU=IT,OU=KEVINLAB-Users,DC=kevinlab,DC=local
distinguishedName: CN=Kevin KA. Admin,OU=IT,OU=KEVINLAB-Users,DC=kevinlab,DC=local
sAMAccountName: kadmin

# Alice AH. HR, HR, KEVINLAB-Users, kevinlab.local
dn: CN=Alice AH. HR,OU=HR,OU=KEVINLAB-Users,DC=kevinlab,DC=local
distinguishedName: CN=Alice AH. HR,OU=HR,OU=KEVINLAB-Users,DC=kevinlab,DC=local
sAMAccountName: ahr

# Frank FF. Finance, Finance, KEVINLAB-Users, kevinlab.local
dn: CN=Frank FF. Finance,OU=Finance,OU=KEVINLAB-Users,DC=kevinlab,DC=local
distinguishedName: CN=Frank FF. Finance,OU=Finance,OU=KEVINLAB-Users,DC=kevinlab,DC=local
sAMAccountName: ffinance

# Sam SS. Sales, Sales, KEVINLAB-Users, kevinlab.local
dn: CN=Sam SS. Sales,OU=Sales,OU=KEVINLAB-Users,DC=kevinlab,DC=local
distinguishedName: CN=Sam SS. Sales,OU=Sales,OU=KEVINLAB-Users,DC=kevinlab,DC=local
sAMAccountName: ssales

# CLIENT01, Workstations, KEVINLAB-Computers, kevinlab.local
dn: CN=CLIENT01,OU=Workstations,OU=KEVINLAB-Computers,DC=kevinlab,DC=local
distinguishedName: CN=CLIENT01,OU=Workstations,OU=KEVINLAB-Computers,DC=kevinlab,DC=local
sAMAccountName: CLIENT01$

# search reference
ref: ldap:///ForestDnsZones.kevinlab.local/DC=ForestDnsZones,DC=kevinlab,DC=local
```

One Step Toward Independence

Question: Can Alice use LDAP to discover the security groups in the domain?

Command: **ldapsearch -x -H ldap://192.168.1.236 -D 'ahr@kevinlab.local' -W -b 'DC=kevinlab,DC=local' '(objectClass=group)' name distinguishedName**

ANSWER: She can!

For specifically security groups:

Command: **ldapsearch -x \ -H ldap://192.168.1.236 \ -D 'ahrn@kevinlab.local' \ -W \ -b 'DC=kevinlab,DC=local' \ '(&(objectClass=group)(groupType:1.2.840.113556.1.4.803:=2147483648))' \ name sAMAccountName distinguishedName groupType member**